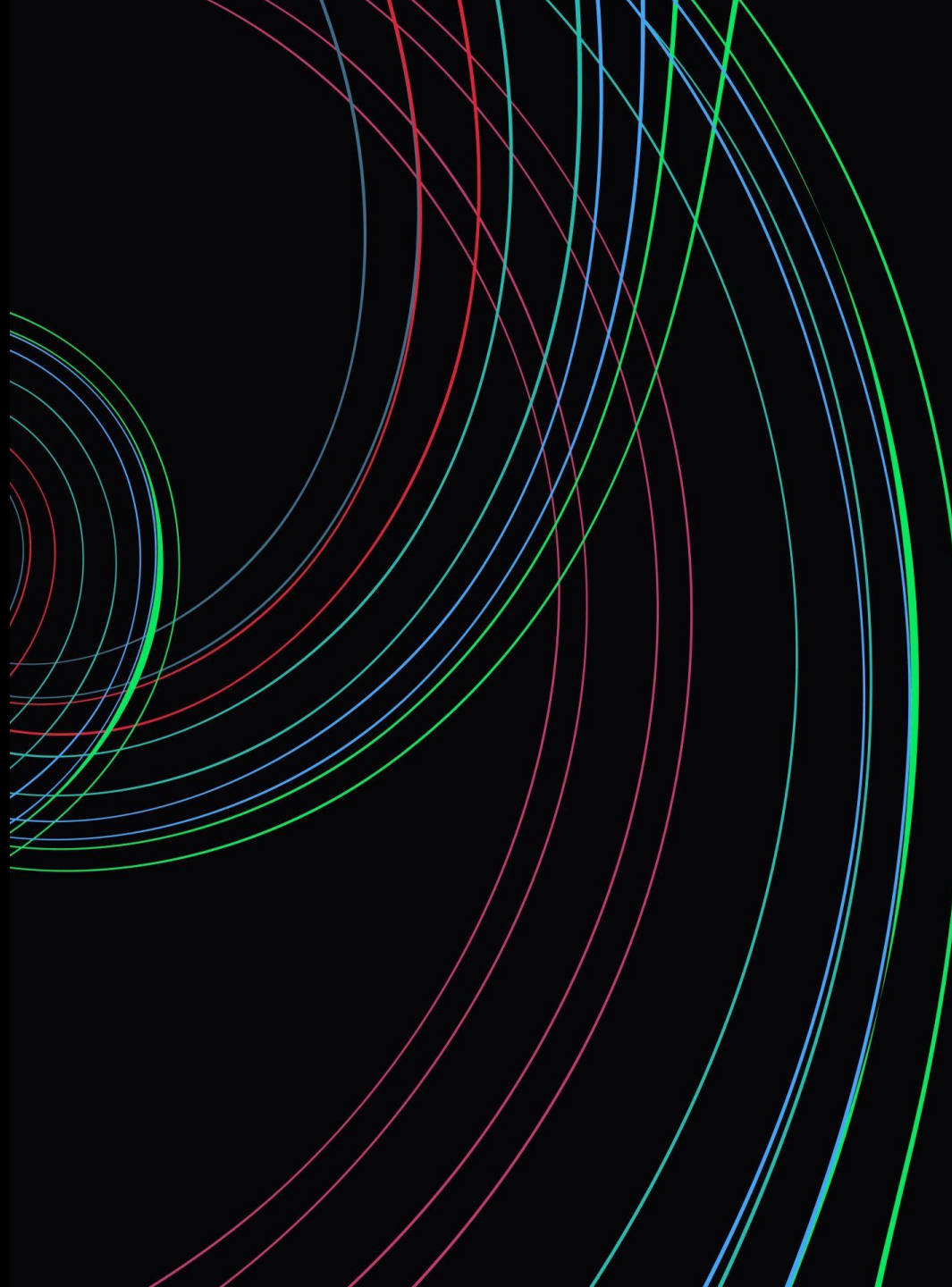

EXPLORING ASE THROUGH FIVE DEMONSTRATIO N SCRIPTS FOR HG ADSORPTION ON AU(111)

Smirnova Elizaveta



SCRIPT 1 – LENNARD-JONES BASELINE

- A 3×3 Au(111) slab with 4 layers
- Lennard-Jones potential with explicit parameters for Hg-Au interaction
- Tests four different adsorption sites: ontop, fcc, hcp, and bridge
- Calculates adsorption energy for each site

```
# Added explicit LJ parameters for Hg-Au interaction
```

```
slab.calc = LennardJones(epsilon=0.05, sigma=2.8, rc=10.0)
```

```
# Added loop over multiple adsorption sites
```

```
sites = ['ontop', 'fcc', 'hcp', 'bridge']
```

for site in sites:

```
    add_adsorbate(slab_ads, 'Hg', height=2.2, position=site)
```

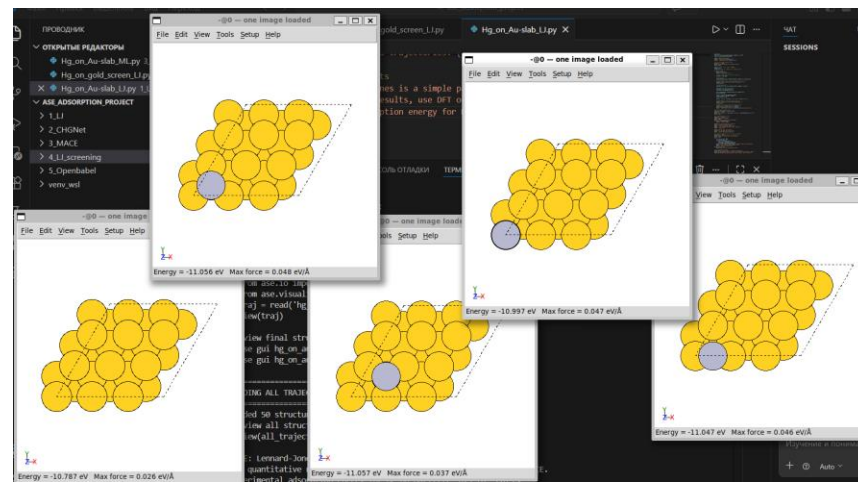
```
Optimizing clean slab...

Reference energies:
Clean slab: -10.7872 eV
Isolated Hg atom: 0.0000 eV

=====
Testing Hg adsorption on different sites
=====
ontop  site: E_total = -8.2954 eV, E_ads =  2.4918 eV
fcc    site: E_total = -10.8787 eV, E_ads = -0.0915 eV
hcp    site: E_total = -10.8788 eV, E_ads = -0.0916 eV
bridge site: E_total = -10.7357 eV, E_ads =  0.0515 eV

-----
Most stable site: hcp with E_ads = -0.0916 eV
-----

NOTE: Lennard-Jones is a simple pair potential.
For quantitative results, use DFT or ML potentials like CHGNet or MACE.
Experimental adsorption energy for Hg on Au(111): -0.5 to -1.0 eV
```



SCRIPT 2 – MACHINE LEARNING WITH CHGNET

Added more sites

```
sites = ['fcc', 'hcp', 'ontop', 'bridge']
```

Added trajectories

```
dyn = BFGS(atoms, trajectory=f'opt_{site}.traj')
```

Added GPU support

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Added database storage

```
db = connect('hg_on_au.db')
```

```
db.write(slab_ads, energy=e_ads, site=site)
```

SCRIPT 3 – MACE POTENTIAL

Different MACE models

```
calc = mace_mp(model="medium", device='cpu')  
# calc = mace_mp(model="large", device='cpu', dtype='float64')
```

Force convergence criteria

```
dyn.run(fmax=0.01) # tight  
# dyn.run(fmax=0.05) # looser
```

Different slab builders

```
from ase.build import bcc111, hcp0001
```

SCRIPT 4 – AUTOMATED SITE SCREENING WITH LJ

Site detection improvements

```
nl = NeighborList([bridge_cutoff] * len(slab), ...)
```

Parallel processing

```
from multiprocessing import Pool
```

```
with Pool() as pool:
```

```
    results = pool.map(run_site, site_items)
```

Comprehensive visualization

```
view(slab_with_ads) # interactive
```

```
write(f'png_outputs/{name}.png', slab_with_ads) # saved images
```

SCRIPT 5 – TRANSITION FROM OPEN BABEL TO ASE

Before (Open Babel)

```
ff = pybel._forcefields["uff"]
```

```
e_slab = ff.Energy()
```

After (ASE)

```
from ase.calculators.emt import EMT
```

```
slab.calc = EMT()
```

```
e_slab = slab.get_potential_energy()
```

Added equation of state

```
from ase.eos import EquationOfState
```

```
eos = EquationOfState(volumes, energies)
```

```
v0, e0, B = eos.fit()
```

KEY TAKEAWAYS FROM ALL FIVE SCRIPTS

- Small changes matter – LJ parameters completely change results
 - ML potentials are worth it – CHGNet/MACE give experimental accuracy
 - Automation is powerful – 96 sites tested automatically
 - Visualization is essential – ase gui and view() catch mistakes
 - Databases organize everything – ase.db for thousands of calculations
-